

bigt.ai

P2P Settlement — Cross-Rail Swap with AI Verification

What this is. A peer-to-peer swap between two rails: crypto on **Bigtangle L0**

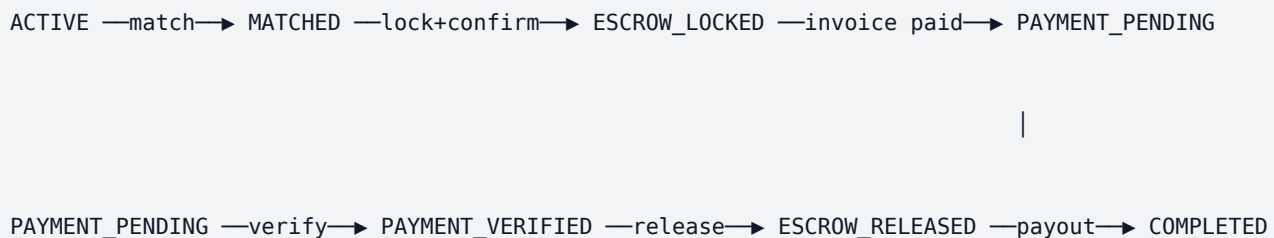
and fiat on **PayPal**. The seller escrows tokens in a 2-of-3 P2SH address (seller, buyer, engine); the buyer pays an exact-amount PayPal invoice; the engine verifies the payment and the escrow releases. There is ****no manual "Confirm" button**** — the agent runs the state machine and every irreversible step (creating an obligation, matching, moving funds) asks you for approval.

How you use it. You talk to dai, your agent, in the **Agent** tab. This guide is captured from the local demo build: the deterministic dev model is driven with `tool: <name> {json}` messages, which is what you see in the screenshots.

Against a deployed region you just describe the trade in words — "sell 1 USDT for 1.01 USD via PayPal" — and the agent picks the tools itself.

Demo vs. live. Screenshots come from the demo build: the engine runs in mock-PayPal mode (`sandbox.paypal.test` invoice URLs, synthetic transaction hashes) and both roles are signed with local env keys. The state machine, signatures, database writes and on-chain anchoring are the real thing; only the PayPal calls and the L0 broadcast are stubbed.

The flow at a glance



#	Step	Who acts	Approval ?
1	Seller posts a signed limit order	seller	yes
2	Buyer matches (receive address + PayPal account)	buyer	yes
3	Seller funds the 2-of-3 escrow, engine proves the lock	seller + engine	no
4	Engine issues an exact-amount PayPal invoice	engine	no
5	Buyer pays the hosted invoice	buyer	no
6	Engine verifies the payment	engine	no
7	Engine + buyer co-sign the release, tokens move	engine + buyer	yes
8	Engine pays the seller via PayPal Payouts	engine	yes
9	Every transition is anchored as a social.p2p-swap record	engine	no

1. The seller lists USDT

Ask the agent to sell. The first thing that happens is an **approval card** on

the Board: creating a sell order commits the seller, so the agent proposes it

and waits.

Agent wants to run p2p_order_create

```
{  
  "giveToken": "USDT",  
  "giveAmount": "1.9971561",  
  "wantAmount": "1.01",  
  "wantCurrency": "USD"  
}
```

Approve

Reject

Approve, and the engine creates the order. The agent reports the order id and

the locked terms:

Chat

approved

you · 8s

tool: p2p_order_create
{
 "giveToken": "USDT",
 "giveAmount": "1.9971561",
 "wantCurrency": "USD",
 "wantAmount": "1.01"
}

Agent · 3s

order ord-973b6fa274c3701e ACTIVE: sell 1.9971561 USDT for 1.01 USD via paypal

message Agent (did:key:...8ge4)

Send

The order is now **ACTIVE** in the engine. The rate is locked at match time, and

the invoice amount is fixed from that moment – the crypto and fiat legs are

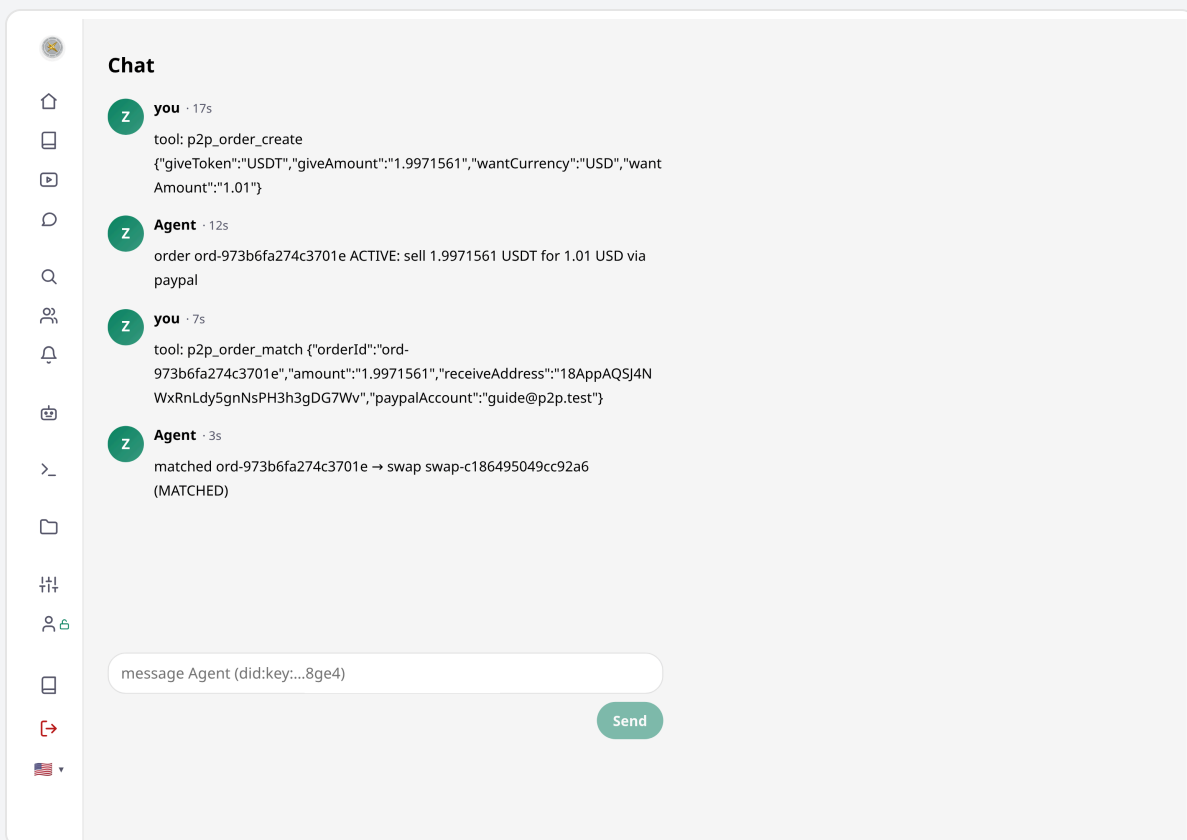
denominated consistently.

2. The buyer matches

The buyer supplies the address that should receive the released tokens and the

PayPal account used for the invoice. Matching is again an approval (it commits

the buyer to pay):

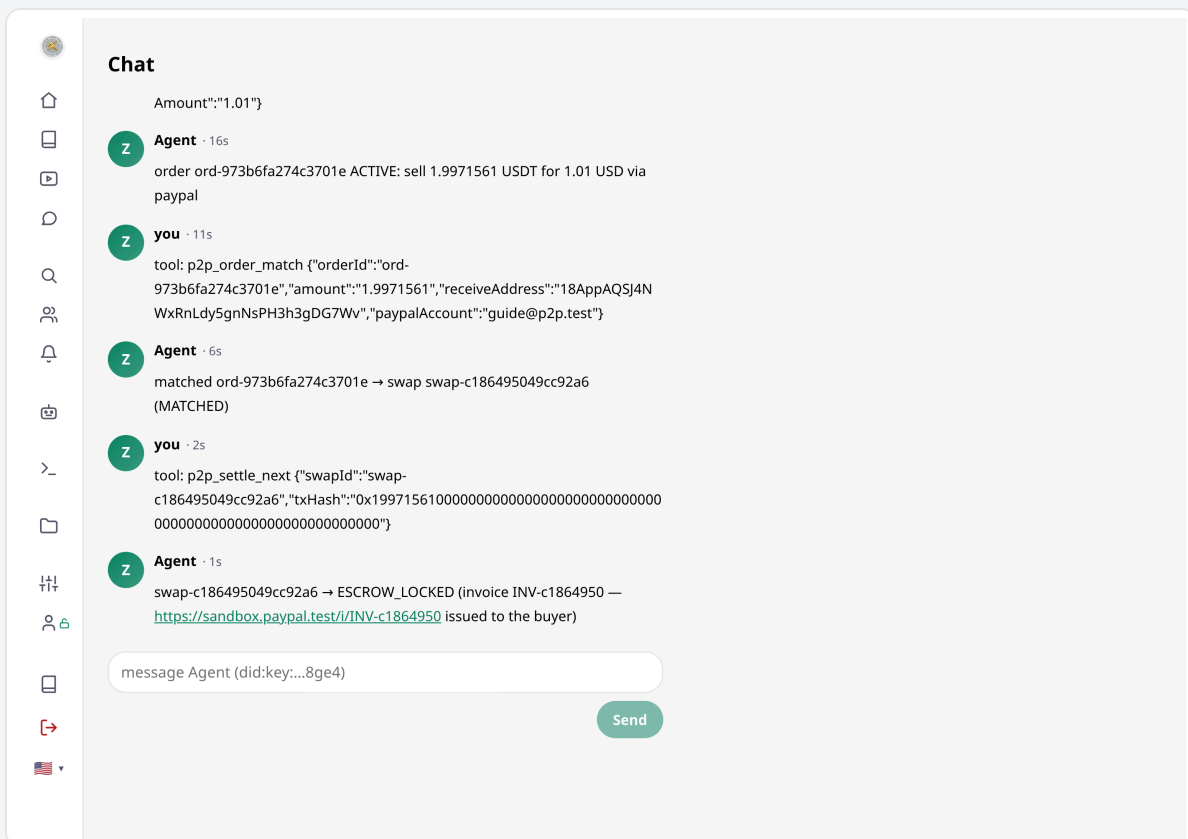


The engine creates a unique `swapId` for the lifecycle and moves the swap to

MATCHED .

3. Escrow lock – the seller funds L0

The seller transfers the tokens to the escrow address, and the agent advances the state machine with the transfer's `txHash`. The engine proves the lock from the chain (`CONFIRMED`, destination `escrowAddress`, amount) and fails closed if any check does not hold. As soon as the lock is proven, the engine issues the buyer an **exact-amount invoice**:



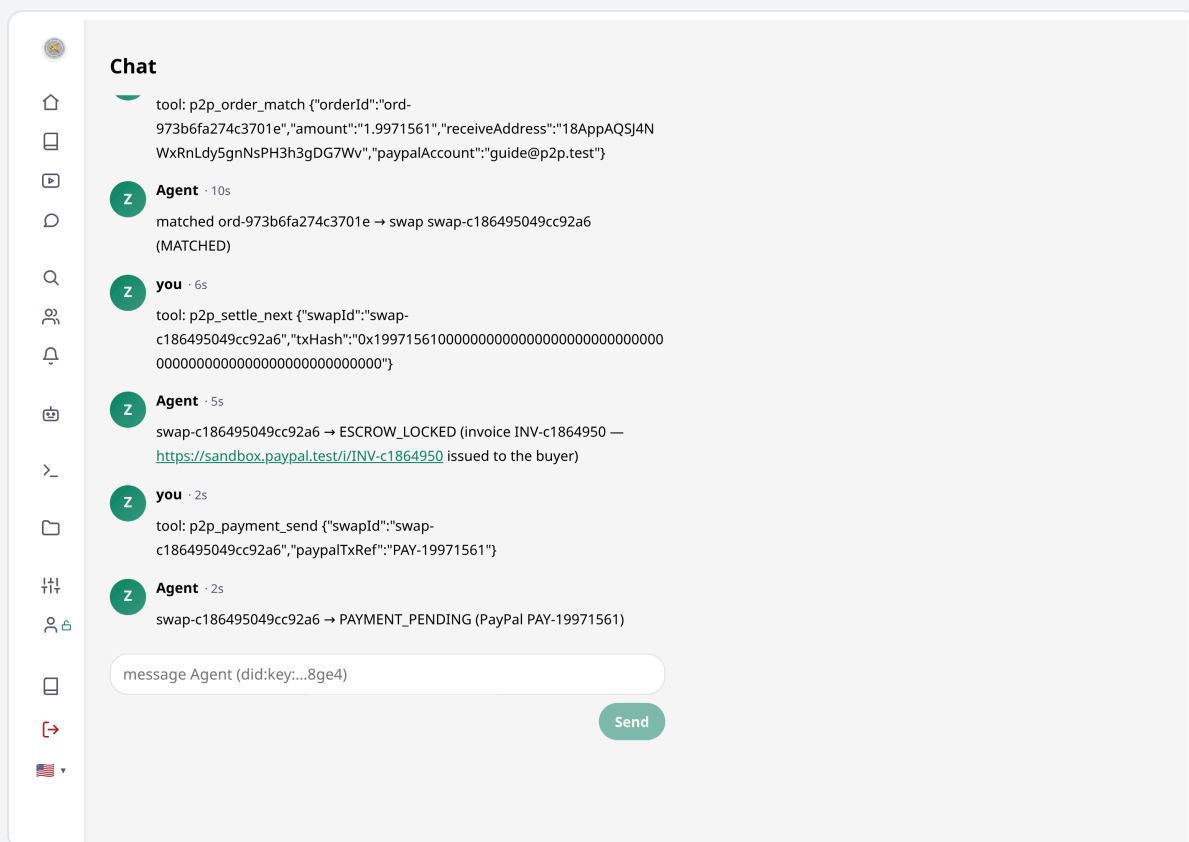
The buyer sees the funds secured before sending any fiat. No approval is needed here: this step only registers evidence and creates the payment request.

4. The buyer pays the invoice

The buyer pays the hosted invoice (PayPal holds the money – the engine never

does), then reports the PayPal reference. This is a *hint*; the real

verification is the engine's PayPal webhook:



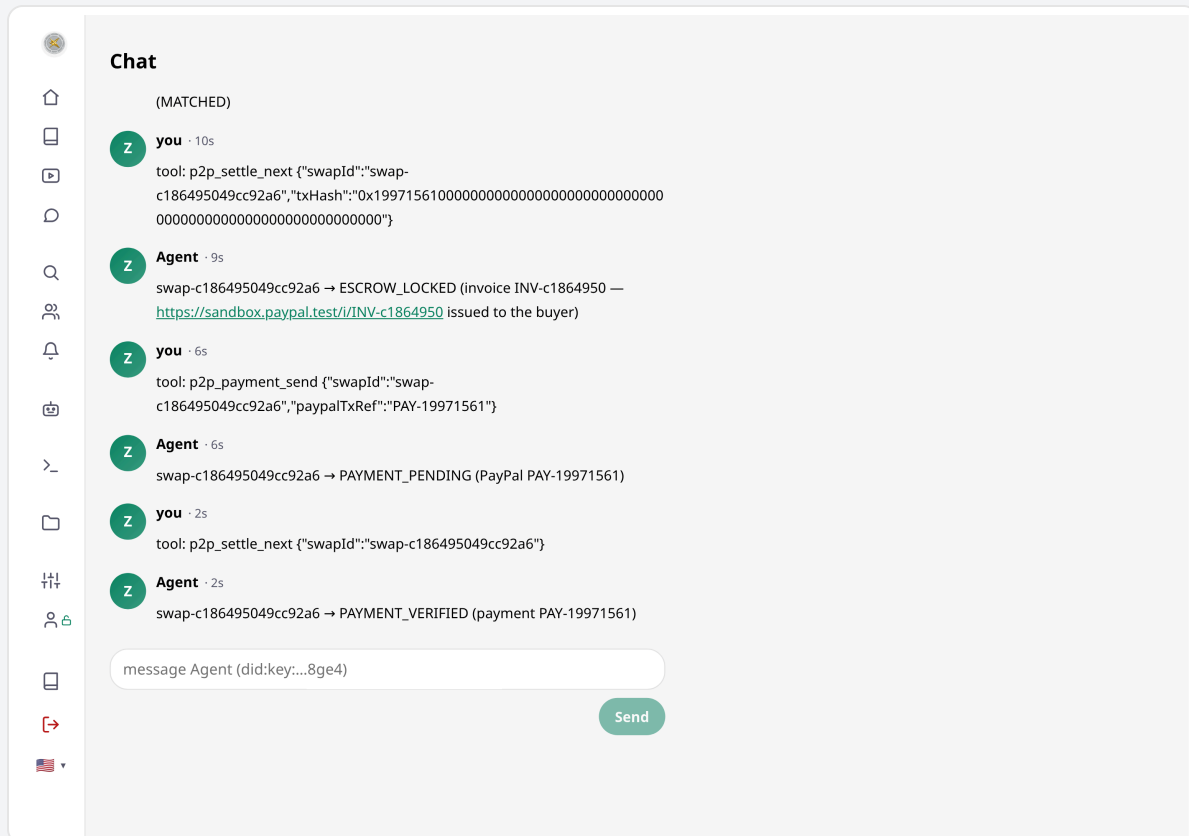
Status: `PAYMENT_PENDING` . A timeout timer starts; if the invoice is never paid,

the seller can expire and refund the escrow without buyer consent.

5. The engine verifies the payment

No seller confirmation is involved. The agent advances the machine and the

engine verifies the payment from its own evidence:



Status: `PAYMENT_VERIFIED` . The escrow is ready to release to the buyer's

```
receive address.
```

6. Release – the funds move

Release moves the escrowed tokens, so it is approval-gated. The agent asks:

[illegible]

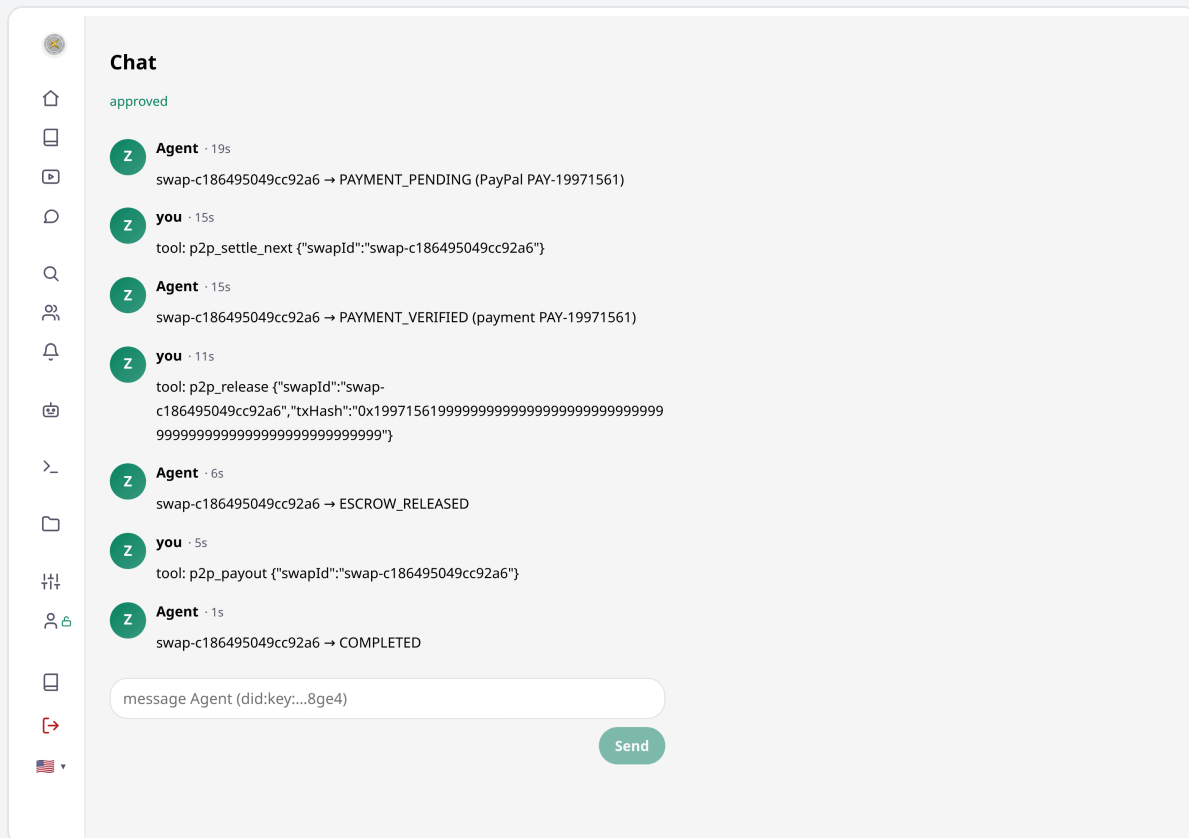
Reject

```
swap reaches  ESCROW_RELEASED :
```

7. Payout – the seller gets the fiat

The final step pays the seller through PayPal Payouts. It moves money, so it is

approval-gated too. With `APPROVE` , the swap reaches `COMPLETED` :



The payout outcome (SUCCESS / FAILED / HELD / ONHOLD) arrives from PayPal's

```
webhook, or is polled as a fallback; a failure can be retried from COMPLETED
```

without redoing the trade.

8. Check the state machine at any time

Ask the agent for the swap status – every transition is one append-only entry

social.p2p-swap record:



Risk	Mitigation
Counterparty walks away	Funds sit in a 2-of-3 P2SH address; nobody moves them alone
Under- or over-payment	The invoice is exact-amount, so it either pays in full or stays unpaid
Fake payment claim	The engine verifies the payment itself – the payer cannot self-assert it

Engine goes rogue	Engine-only steps require the engine DID signature; every transition is anchored and publicly auditable
Chargeback after release	PAYMENT.CAPTURE.REVERSED sets a freeze: forward steps stop, refund/expire stay reachable
Dispute	CUSTOMER.DISPUTE.* pauses the swap until it resolves
Payout failure	HELD / FAILED / BLOCKED are first-class states; retry with the same payout reference

Approvals in the demo

Tool	Approval	Why
p2p_order_create	yes	commits the seller to the terms
p2p_order_match	yes	commits the buyer to pay
p2p_settle_next	no	evidence-only steps (lock, verify)
p2p_payment_send	no	a payment <i>hint</i> ; the engine verifies independently
p2p_release	yes	moves funds
p2p_payout	yes	moves funds
p2p_status	no	read-only

Going live

The demo runs against the in-repo settlement service with mock PayPal. To run

the real flow you need a PayPal business account with Payouts activated, its

API credentials and webhook id, and the escrow signer wired to the L0

broadcast step. Until then the engine runs the same state machine without

touching real money.

